

Building Effective Agents

What an agent *actually* is, the seven building-block patterns, and the rules for picking the simplest one that solves your problem.

Based on: anthropic.com/engineering/building-effective-agents

Learning Objectives

- Distinguish **workflows** (LLMs on predefined code paths) from **agents** (LLMs that direct their own work) and explain why the distinction matters.
- Recognize the foundational **augmented LLM** building block (retrieval + tools + memory) underneath every other pattern.
- Identify the five workflow patterns — *prompt chaining*, *routing*, *parallelization*, *orchestrator-workers*, *evaluator-optimizer* — and pick the right one.
- Decide when full agent autonomy is justified and when a simpler workflow is the better engineering choice.
- Apply the agent-computer interface (ACI) principles when designing tools your agent will actually use well.

1. Workflows vs. Agents — The Core Distinction

The word "agent" gets used loosely to mean anything from a chatbot to a fully autonomous robot. Anthropic narrows it down to one specific axis: **who decides what step happens next?**

EVERYDAY ANALOGY

Before: The difference between a vending machine and a personal shopper. The vending machine has a fixed flow — every step is wired in. A personal shopper hears your goal and decides themselves which stores to visit, what to compare, when to call you for clarification, when to stop.

The pain: Most teams call *both* systems "AI agents" the moment Claude is involved. That hides the actual engineering choice.

The mapping: Workflows are vending machines: LLMs and tools wired together by code paths *you* wrote. **Agents** are personal shoppers: LLMs that read the goal and dynamically pick their own next step. Both are useful — conflating them leads to over-engineered prototypes and under-engineered production systems.

TECHNICAL DEFINITION

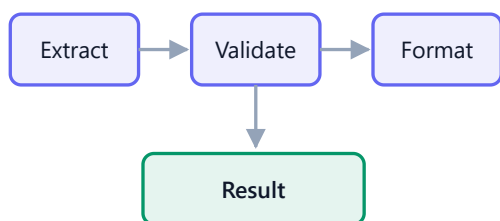
Workflows are systems where LLMs and tools are orchestrated through predefined code paths. The structure is static. You can read the source and predict every API call.

Agents are systems where LLMs dynamically direct their own processes and tool usage. You give the model a goal and a toolbox; it decides which tool to call, in which order, and when to stop. The structure of the run is decided at runtime.

Most production systems combine both — a workflow at the top level that calls an agent for the genuinely open-ended sub-task, or an agent that internally invokes deterministic workflows as "tools."

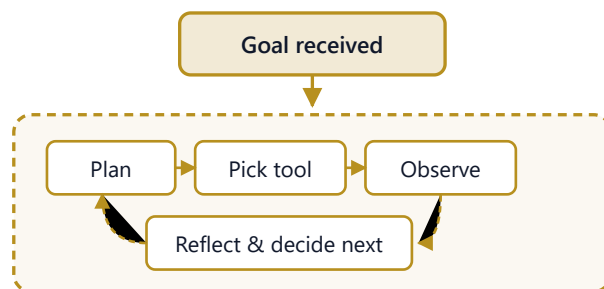
FIXED PATH VS. SELF-DIRECTED LOOP

WORKFLOW · PREDEFINED PATH



Every run takes the same path.

AGENT · SELF-DIRECTED LOOP



Path is decided at runtime by the model.

WHY IT MATTERS

Picking the wrong side of this line is the most common cause of failed AI projects. Teams reach for "an agent" when their problem is actually a 3-step workflow — and end up with a system that costs 10× more, runs 5× slower, and is impossible to debug. Others build a rigid 50-step workflow when an agent could have handled the variety in 8 lines of code. Anthropic's recommendation: *start simple, escalate complexity only when needed.*

When do you actually need the agent end of the spectrum? Anthropic's most quoted — and most ignored — recommendation:

2. When (and When Not) to Use Agents

The article opens with a recommendation that's easy to skip past: *find the simplest solution possible, and only increase complexity when needed*. It then drives the point home with a sharper version: "This might mean not building agentic systems at all." Coming from the company that sells the model, that's notable. Take it seriously.

The reasoning is grounded in tradeoffs. Agentic systems trade **latency and cost** for **better task performance on open-ended problems**.

IF YOUR TASK IS...	THEN START WITH...	WHY
A single LLM call solves it	Just an LLM call	Optimize the prompt with examples and retrieval before adding orchestration.
Decomposable into fixed sub-steps	Prompt chaining	Each step is small and validatable. Buy accuracy with a tiny bit of latency.
Distinct categories needing different handling	Routing	One specialized prompt per category beats one generalist prompt.
Multiple independent sub-tasks or perspectives	Parallelization	Speed (sectioning) or robustness via voting.
Sub-tasks unknowable in advance	Orchestrator-workers	Let the model decompose at runtime; structure the rest.
Output quality has clear evaluation criteria	Evaluator-optimizer	Iterative refinement when feedback is measurable.
Open-ended, # of steps unknowable, sandboxed	Full agent	Worth the cost when nothing simpler fits.

COMMON MISCONCEPTIONS

- "An agent is more powerful, so it's always better." — No. An agent solving a problem a routing workflow could solve is just an unreliable, expensive, slow version of the same answer.
- "If we use a framework, we don't need to think about this." — Frameworks (Agent SDK, LangChain, etc.) make it easy to *start* but often add abstractions that obscure prompts and make debugging painful. Prefer LLM APIs directly until the abstraction earns its weight.
- "Agents and chatbots are the same thing." — A chatbot generates responses. An agent uses tools in a loop to take actions in the world. The difference is whether *side effects* happen mid-conversation.

THE COST / LATENCY REALITY

A single LLM call: 1 API request, ~1–3 seconds, ~\$0.003 (Sonnet). A 3-step prompt chain: 3 calls, ~3–9 seconds, ~\$0.009. An agent that loops 10 times: 10 calls, 30+ seconds, ~\$0.030 — and unbounded if it doesn't converge. **The simplest pattern that works wins.** Reach for the next pattern only when the simpler one demonstrably fails on your eval set.

3. The Augmented LLM — The Foundation Block

Every pattern in the article — chaining, routing, agents, all of them — is built out of one repeating piece: an LLM augmented with three capabilities: **retrieval, tools, and memory**. Don't think of the LLM as a chatbot, think of it as a reasoning engine with three plug-in slots.

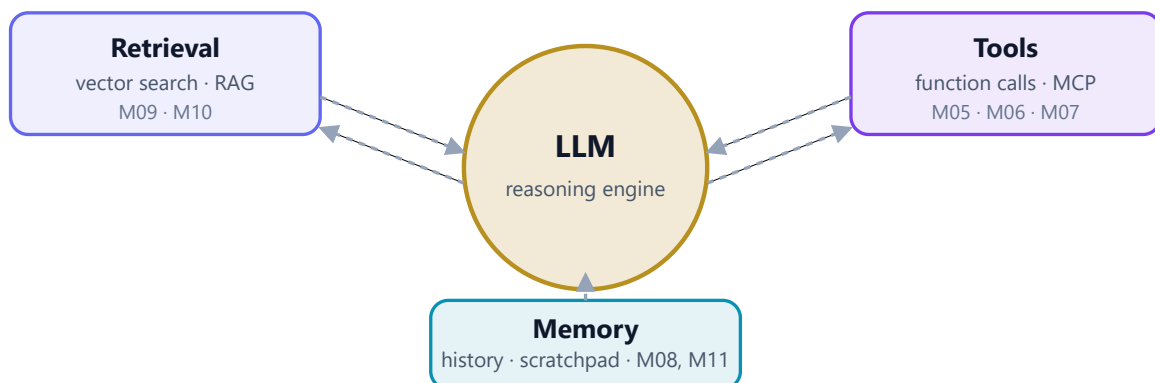
EVERYDAY ANALOGY

Before augmentation: A bare LLM is like a brilliant new hire on day one. Smart and well-read but doesn't know your codebase, can't open Jira, and forgot what you discussed yesterday.

The pain: A bare LLM hallucinates because it has no way to look things up, no way to act on the world, and no continuity across sessions. Treating that bare LLM as your agent is the source of most failures.

The mapping: The augmented LLM is the same hire on day 30 — wiki access (retrieval), can run scripts and update tickets (tools), remembers the past month of decisions (memory). Every pattern is a way of *arranging* these augmented LLMs.

THE AUGMENTED LLM — THREE PLUG-IN SLOTS



WHY IT MATTERS

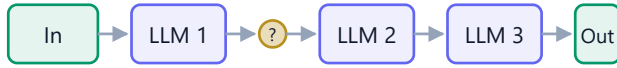
Most production agent failures aren't reasoning failures — they're augmentation failures. The retrieval pulled stale data. The tool returned an error the model didn't know how to interpret. The memory truncated the wrong message. Every "Claude hallucinated" bug ticket is usually a "we never gave Claude what it needed" bug ticket.

With the building block in hand, here are the five workflow patterns the article catalogs — in roughly increasing complexity.

4. The Six Pattern Topologies at a Glance

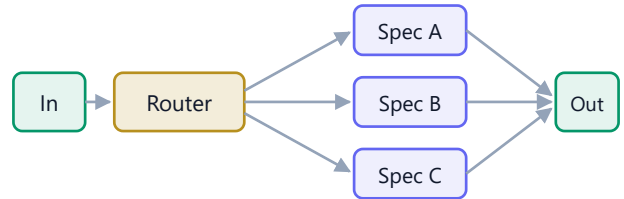
Each pattern is a different shape for arranging augmented LLMs into a useful system.

1 · PROMPT CHAINING



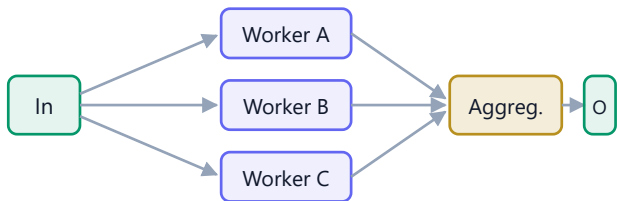
Sequential, with optional gates between calls.

2 · ROUTING



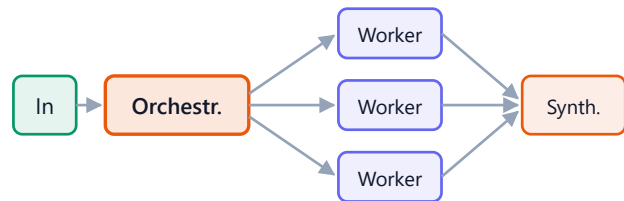
Classify, then dispatch to a specialist.

3 · PARALLELIZATION



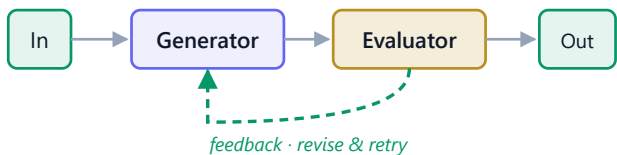
Concurrent for speed (sectioning) or voting.

4 · ORCHESTRATOR-WORKERS

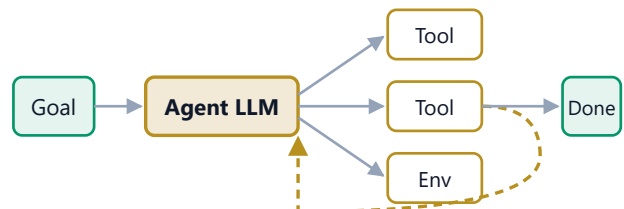


Like routing, but workers picked at runtime.

5 · EVALUATOR-OPTIMIZER



6 · AGENT



P1 Prompt Chaining

EVERYDAY ANALOGY

Before chaining: Asking a copywriter to "write a marketing email, translate it to French, and verify the translation" all at once. They'll do all three, but a fluent French speaker would still spot rough patches because the copywriter was juggling three jobs.

The pain: A single LLM call asked to do three things produces work that's mediocre at all three. Worse, you can't inspect the intermediate steps.

The mapping: Prompt chaining is the assembly line. Step 1: write the email. Stop. Inspect. Step 2: translate. Stop. Inspect. Step 3: verify. Each call is focused; the output of one becomes the input of the next; programmatic **gates** sit between steps.

TECHNICAL DEFINITION

Prompt chaining decomposes a task into a sequence of LLM calls where each processes the previous one's output. Optional programmatic checks (*gates*) sit between calls.

Right tool when: The task can be cleanly cut into fixed sub-steps, you know the order, and each sub-step is small enough that the model handles it reliably. Outline-then-write, draft-then-edit, extract-then-summarize.

Wrong tool when: The order of steps depends on the input. That's a routing or orchestrator-workers problem.

WHAT JUST HAPPENED?

Three API calls instead of one and ~5 extra seconds of latency. In exchange, each step is small enough that the model handles it well, the gate kills bad runs early, and when the output is wrong you can pinpoint *which step* went wrong by inspecting intermediate text.

EVERYDAY ANALOGY

Before routing: A general-practitioner doctor handles every complaint — broken leg, weird rash, chest pain. Competent at all of it but excellent at none, always looking up reference material.

The pain: One LLM prompt that has to handle "billing question, technical bug, refund, account deletion" ends up bloated. Performance is mediocre across the board.

The mapping: Routing is triage. A short, fast classifier reads the input and dispatches to the right specialist prompt. Each specialist is short and sharply focused. You can swap in a stronger model only for the harder categories.

TECHNICAL DEFINITION

Routing classifies the input and directs it to a specialized follow-up task. Classification can be an LLM call (fuzzy categories) or a deterministic rule (obvious categories). Each branch has its own prompt, possibly its own model and tool set.

Right tool when: Inputs fall into distinct categories that benefit from different handling. Customer support intents, document types, query difficulties (cheap model for easy, expensive for hard), language detection.

Watch for: Misclassification cascades. Add a fallback branch for low-confidence classifications and log misroutes for tuning.

Cost optimization tip: The classifier uses Haiku (cheap, fast); the specialist uses Sonnet. You only pay for the smarter model on the work that needs it.

EVERYDAY ANALOGY

Before parallelization: A single proofreader reads end-to-end checking grammar, fact accuracy, and tone all at once. By page 80 they're tired, mixing up the passes.

The pain: Several independent jobs done serially in one LLM call wastes wall-clock time and dilutes attention. For safety-critical checks, a single pass is a single point of failure.

The mapping: Two flavors. **Sectioning** hands different chapters to different proofreaders so they finish in parallel. **Voting** hands the *same* chapter to three proofreaders and combines their findings — useful when one careful pass isn't reliable enough.

TECHNICAL DEFINITION

Parallelization runs multiple LLM calls simultaneously and aggregates their outputs.

Sectioning: Independent sub-tasks done at the same time — translate a long document by chapter; validate three different aspects (toxicity, on-topic, factual) in parallel.

Voting: Same task run multiple times with varied prompts/temperature; results combined by majority vote, union, or LLM synthesis. Used for code-vulnerability scans, output safety classifiers, content evaluation where multiple perspectives improve quality.

Watch for: Cost. N calls cost $N \times$ one call. Use parallelization when speed or robustness justifies that cost.

A canonical example: three concurrent safety checks (toxicity, off-topic, PII). All fire concurrently, so wall-clock time is roughly one call — about $3 \times$ faster than sequential. *Fail-closed* default in error handlers is deliberate: if a safety check can't run, treat the post as flagged rather than silently approve.

EVERYDAY ANALOGY

Before: A senior editor receives a vague brief: "produce a comprehensive market briefing on emerging fintech in Southeast Asia." They can't pre-define the chapters — they don't know yet whether the briefing needs three country deep-dives or seven.

The pain: Routing won't work because the categories aren't known up front. Prompt chaining won't work because the steps depend on what we find. The decomposition itself is part of the problem.

The mapping: The senior editor reads the brief, decides on the chapters, hands each to a specialist writer, then weaves the results together. That's orchestrator-workers: a central LLM dynamically decomposes the work, dispatches sub-tasks, and synthesizes.

TECHNICAL DEFINITION

Orchestrator-workers uses one LLM as a dispatcher: it reads the input, generates a list of sub-tasks dynamically, calls worker LLMs (in parallel where possible) to solve each, and synthesizes a final answer.

The key difference from *parallelization*: sub-tasks aren't pre-defined — the orchestrator decides them at runtime. The key difference from a *full agent*: structure is still bounded — orchestrator decomposes, workers solve, orchestrator synthesizes, done. No open-ended loop.

Right tool when: Complex tasks where you can't predict sub-tasks in advance — multi-file code edits, research summaries, analysis pipelines that depend on what the data looks like.

CERT TIP — DOMAIN 1.2

The exam favors orchestrator-workers (a.k.a. coordinator + subagents) over flat or peer-to-peer multi-agent designs. Memorize the four properties: **single coordination point, context isolation between workers, structured result aggregation, auditable decision flow.**

EVERYDAY ANALOGY

Before: A novelist sends each chapter to a publisher and gets a single "looks good" or "please revise." Without specific feedback, every revision is a guess at what the publisher actually wants.

The pain: Many tasks aren't one-shot. Translation, code review, technical writing, summarization — these benefit from a critique pass. Asking the original LLM to "make it better" produces a different version with the same blind spots.

The mapping: Evaluator-optimizer uses two LLM roles in a loop. A **generator** produces a candidate output. An **evaluator** — a separate prompt with a different focus — critiques against explicit criteria. The generator revises using the critique. Loop until approved or max-iterations cap kicks in.

TECHNICAL DEFINITION

Evaluator-optimizer is a generator + critic loop. Generator produces, evaluator scores against criteria, generator revises. Loop terminates when score crosses a threshold or iterations cap.

Right tool when: (1) you can articulate clear evaluation criteria the model can apply, AND (2) iterative refinement provides measurably better outputs. Translation quality, literary editing, code that must pass tests, documents that must satisfy a checklist.

Wrong tool when: "Good" is fuzzy and subjective — the evaluator produces noisy critiques that the generator chases unproductively, burning calls without converging.

EVERYDAY ANALOGY

Before full agents: Every workflow above has a code-defined shape. The path is fixed by the code you wrote.

The pain: Some tasks resist that. "Reproduce this bug, find the root cause, write a fix, run tests, open a PR." You can't pre-decide how many test runs, how many file edits, or what investigation order will be needed — that depends entirely on what the bug turns out to be.

The mapping: A full agent is the LLM running its own loop. You hand it a goal and a toolbox; it picks tools, observes results, decides on its own when the work is done. Path length is unbounded. Cost and latency are unbounded. In exchange, the agent can handle problems whose *shape* couldn't have been pre-determined.

TECHNICAL DEFINITION

An **agent** is an LLM in a tool-calling loop: receive task → pick tool → observe result → decide next step (more tools, or stop) → loop.

Right tool when: Open-ended problems where the number of steps is unknowable in advance, AND you can run in a sandboxed environment, AND you have evals to catch regressions. Canonical examples: coding agents that solve real GitHub issues by iterating against tests, computer-use agents that navigate UIs.

The cost is real. The article warns about three things: **higher costs** (loops mean $N \times$ tokens), **compounding errors** (one wrong step contaminates everything that follows), and **need for guardrails** (sandboxed execution, max-step limits, human-in-the-loop checkpoints for high-stakes actions).

SANDBOXING & GUARDRAILS

The article is explicit: agents need **extensive testing in sandboxed environments** and **appropriate guardrails** before being trusted with autonomy. Production agents add: max-steps caps, max-cost caps, an allowlist of tools per task, human-in-the-loop confirmation for irreversible actions (database writes, emails, deploys), and pervasive logging.

Combining Patterns — Real Systems Mix Them

The article makes one final point: *"These building blocks aren't prescriptive. They're common patterns developers can shape and combine to fit different use cases."* Don't pick exactly one. Real production systems compose them.

Three useful compositions:

- **Routing → (specialized chain or agent):** A top-level router classifies the request, then dispatches to either a deterministic chain (cheap, predictable cases) or a full agent (rare, open-ended cases). Most customer-support stacks look like this.
- **Agent with workflow tools:** Inside an agent, individual "tools" can themselves be deterministic workflows. The agent calls a tool that internally does prompt-chaining to summarize a document — the agent doesn't see that complexity.
- **Orchestrator-workers with evaluator-optimizer per worker:** A research orchestrator spawns subagents, and each subagent runs a generator-critic loop on its draft before returning.

WHY IT MATTERS

The mistake teams make is treating the patterns as competing options — "are we doing routing or are we doing agents?" — when in reality a real system is usually a tree: a workflow at the top, agents at the leaves where the open-ended work lives, and inside each agent, more workflows. The patterns are a vocabulary for talking about those tree levels, not a single choice for the whole system.

Tool Engineering — The Agent-Computer Interface (ACI)

Anthropic devotes a section to a topic that often gets shortchanged: how you design the tools the agent uses. They coin the term **Agent-Computer Interface (ACI)** — the agent's analog of HCI. The takeaway: invest as much effort in the ACI as you'd invest in a UI for a human.

TECHNICAL DEFINITION

The **ACI** is the surface the model sees: tool names, descriptions, parameter schemas, return formats, error messages. The article reports that on internal coding-agent work, tool-design effort "often required more effort than overall prompt development." Translation: better tools beat better prompts, and most teams under-invest here.

The ACI checklist

- **Give the model enough tokens to think before acting.** Don't cap output so tightly that the agent skips reasoning. Allow space for a brief plan before each tool call.
- **Keep formats close to what the model sees naturally on the open web.** JSON, markdown, plain English — not bespoke binary formats or custom DSLs. Models are dramatically better at common formats.
- **Eliminate formatting overhead.** Don't require counting whitespace, escaping exotic characters, or maintaining rigid line numbers. Every char of overhead is one more thing to get wrong.
- **Test extensively with example inputs.** Run the tools against realistic inputs and read the model's tool calls. You'll spot ambiguities the schema didn't catch.
- **Apply *poka-yoke* design** (mistake-proofing). Shape the tool so common errors are impossible. Anthropic's classic example: their coding agent kept generating relative file paths that broke when the working directory shifted, so they redesigned the tool to require absolute paths only — the bug class disappeared.
- **Document the tool the way you'd document it for a junior engineer.** What does it do, what are the inputs, what does it return, what errors can it produce, what should the caller do about each error?

THE ACI INSIGHT

If your agent is misbehaving, the first instinct is to edit the system prompt. The article's data points elsewhere: more often, the fix is in the tool. Rename a confusingly-named parameter. Make a return format unambiguous. Add a missing field to an error response. The same model on the same prompt with better tools performs dramatically better. ACI is the highest-leverage intervention most teams ignore.

Real Applications — Where These Patterns Show Up

The article's appendix grounds the abstract patterns in two well-developed real applications.

Customer Support

Customer support combines a chatbot interface with tool integration to access customer data, history, and account systems — and to take actions like issuing refunds. A strong fit because:

- The interaction is naturally conversational (LLMs are good at this).
- Success is *measurable*: did the issue get resolved? Was the customer satisfied?
- Pricing per resolution maps cleanly to value — outcome-based pricing actually works.

The architecture is typically routing-at-the-top (intent classification) plus a constrained agent in each lane that has access to relevant tools — refund tools for billing, account tools for accounts. Pure free-form agents are usually overkill.

Coding Agents

Coding agents (the SWE-bench category) are the article's flagship example of when full agent autonomy is justified. The reasoning:

- Solutions are **verifiable through automated tests** — the agent doesn't need a human in the loop on every step because the test suite is the ground truth.
- The agent can **iterate using test results as feedback**: try a fix, run tests, see what failed, try a refined fix.
- The problem space is genuinely open-ended — the agent might need to read 3 files or 30, edit 1 line or rewrite a module — impossible to pre-script.

Even here, the article tempers the autonomy: "*human review remains crucial for ensuring solutions align with broader system requirements.*" The PR review step is the human guardrail. The agent works inside a sandbox; humans gate what gets merged.

THE COMMON THREAD

Both examples share one thing: a clear, automatic feedback signal. Customer support has resolution outcomes. Coding has test results. When that signal exists, agents become viable. When it doesn't — when "good" can only be judged by a human reading the output — you're probably looking at a workflow problem, not an agent problem.

Module Summary — Key Concepts

- **Workflows vs. agents:** The line is who decides the path. Workflows: predefined code paths you wrote. Agents: LLM dynamically directs itself.
- **Start simple:** A bare LLM call beats a chain. A chain beats an agent. Reach for the next pattern only when the simpler one provably fails.
- **Augmented LLM:** The reusable building block under everything. Retrieval + tools + memory, with each capability tailored to the use case.
- **Five workflow patterns:** Prompt chaining (sequential), routing (classify and dispatch), parallelization (concurrent for speed or voting), orchestrator-workers (runtime decomposition), evaluator-optimizer (generator + critic loop).
- **Full agents:** Worth the cost only for open-ended problems where steps are unknowable in advance and you can sandbox + add guardrails. Customer support and coding agents are the canonical fits.
- **ACI:** Tool design is the highest-leverage intervention most teams ignore. Better tools beat better prompts.
- **Combining patterns:** Real systems are trees — a workflow at the top, agents at the leaves, more workflows inside the agents.

How This Maps to the Course

ARTICLE CONCEPT	COURSE MODULE(S)
Augmented LLM — retrieval	M09, M10
Augmented LLM — tools / MCP	M05, M06, M07
Augmented LLM — memory	M08, M11
Prompt chaining	M03, M03B
Routing & orchestrator-workers	M14, M15B
Evaluator-optimizer	M17, M18
Agents	M12, M13
Sandboxing & guardrails	M15, M16, M17
ACI / tool engineering	M05, M07, M26

Where to go next: Pick the pattern that matches your current project and dive into the corresponding course module. Most teams overestimate how much agent autonomy they need. Try the simpler pattern first — you can always escalate when your eval set proves it's necessary.